JJBike

1. Objective.

To build a data warehouse so that the administrator of the fictional company JJBike can understand their business.

2. Modeling.

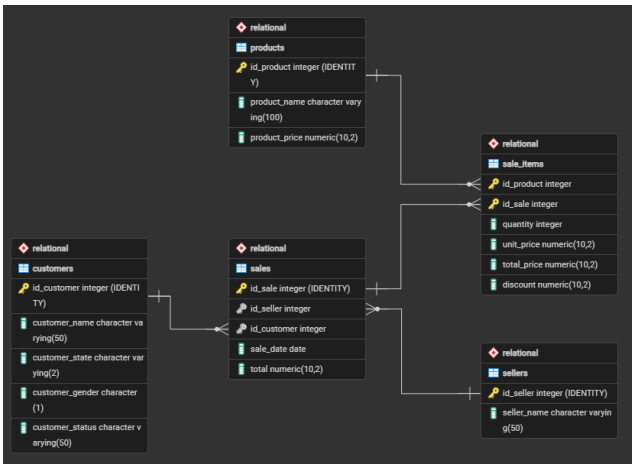
To create the analytical database structure model (OLAP) based on the transactional database (OLTP), using the Star Schema model (the most common model for Business Intelligence), the following transformations were made:

1. To create the analytical database structure model (OLAP) based on the transactional database (OLTP), the subject to be analyzed (the fact) and the dimensions are the various perspectives from which the fact is analyzed (the dimensions), using the Star Schema model (the most common model for Business Intelligence), the following transformations were made:

Dimension	Attributes	Source Table
dim_product (what?)	product_key (SK)	-
	id_product	product
	product_name	product
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_seller (who?)	seller_key (SK)	-
	id_seller	seller
	seller_name	seller
	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_customer (who?)	customer_key (SK)	-
	id_customer	customer
	customer_name	customer
	customer_state	customer
	customer_gender	customer
	customer_status	customer

	validity_start_date (versioning)	-
	validity_end_date (versioning)	-
dim_time (when?)	time_key (SK)	-
	date	-
	time_day	-
	time_month	-
	time_year	-
	time_weekday	-
	time_quarter	-

Fact	Consolidated Table	Granularity	Attributes	Source Table
fact_sales	sales	Product item per sale.	sale_key (SK)	-
			seller_key (FK)	dim_seller
			customer_key (FK)	dim_customer
			product_key	dim_product
			time_key (FK)	dim_time
			quantity	sale_items
			unit_price	sale_items
			total_price	sale_items
			discount	sale_items



3. pgAdmin 4.

In pgAdmin 4, the 'JJBIKE' database was created.

Note: SQL code development will follow the Snake Case naming convention.

4. Transactional Environment (OLTP) – pgAdmin 4.

4.1. Schema and Transactional Table Creation.

The Relational schema was created, serving as the source of transactional data for the JJBike company. Within it, five operational tables were created using the file 1. Scripts_Create_Table_Relational.sql:

- **sellers:** Stores seller data. Its primary key, id_seller, is automatically generated by the GENERATED ALWAYS AS IDENTITY clause. The seller_name attribute has a NOT NULL constraint, ensuring no seller is registered without a name;
- **products:** Stores the product catalog. The primary key id_product is automatically generated. The product_name and product_price attributes are defined as NOT NULL, ensuring every product has a name and price recorded;
- **customers:** Stores customer records. The primary key id_customer is automatically generated. The customer_name, customer_state, customer_gender, and customer_status attributes are all NOT NULL, ensuring each customer profile is always complete;
- **sales:** Represents the header of each sale. The primary key id_sale is automatically generated. The id_seller and id_customer columns are NOT NULL Foreign Keys that reference the sellers and customers tables respectively, preventing a sale from being recorded without an associated seller or customer. The sale_date attribute records the date the sale occurred, and total stores the transaction total, both mandatory;
- **sale_items:** N:M junction table between sales and products. Its primary key is composed of id_product and id_sale. As a junction table, two distinct referential integrity constraints were applied:
    - ON DELETE RESTRICT on id_product: prevents the deletion of a product in Relational.products while it is referenced in any sale item, preventing historical reports from losing the sold product description;
    - ON DELETE CASCADE on id_sale: ensures that when a record in Relational.sales is deleted, all its corresponding items in sale_items are automatically removed, eliminating orphan records from non-existent sales;
    - The attributes quantity, unit_price (unit price), total_price (total item value), and discount (applied discount) are all NOT NULL, ensuring the integrity of the metrics that will subsequently feed the fact table.

The GENERATED ALWAYS AS IDENTITY clause was used in all tables for automatic Primary Key generation. All columns, except the Primary Keys themselves and validity_end_date in the dimensions, were defined with the NOT NULL constraint, ensuring data integrity at the database level. The default constraint when defining a FK without specifying a deletion or update action is NO ACTION, which blocks the operation and generates an error if an attempt is made to delete a parent record with dependents in the child table, preventing the creation of orphan records.

4.2. Data Insertion into Transactional Tables.

After the tables were created, data insertions were performed using the files 2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql, and 6. Insert_Into_SaleItems.sql. Each file executes INSERT INTO commands into the respective table of the Relational schema. Since the Primary Keys are managed by GENERATED ALWAYS AS IDENTITY, the scripts do not explicitly provide these values; the database assigns them automatically and sequentially to each inserted record.

5. Analytical Environment (OLAP) – pgAdmin 4.

5.1. Schema and Dimensional Table Creation.

The Dimensional schema was created, representing the analytical environment (Data Warehouse) structured in the Star Schema model. Within it, the dimensions dim_seller, dim_customer, dim_product, dim_time, and the fact table fact_sales were created using the file 1. Scripts_Create_Table_Dimensional.sql.

The GENERATED ALWAYS AS IDENTITY clause was used to automatically generate the Surrogate Keys (SKs) for all dimensions and the fact table, serving as the internal primary key of the Data Warehouse. All columns were defined with NOT NULL, except for the Primary Keys themselves and the validity_end_date field in dimensions with SCD Type 2; this field remains NULL while the record is active, being populated only when a new version of the record is created.

The dimensional tables have the following attribute structure:

- **dim_seller:** seller_key (automatically generated SK, internal DW PK), id_seller (source ID from the Relational.sellers table, preserved for traceability), seller_name (seller name), validity_start_date (date this record became valid), and validity_end_date (date this record was closed; NULL indicates an active record);
- **dim_customer:** customer_key (automatically generated SK, internal DW PK), id_customer (source ID), customer_name, customer_state, customer_gender, customer_status, validity_start_date, and validity_end_date. As it has multiple trackable attributes, it is the dimension with the greatest SCD Type 2 versioning potential;
- **dim_product:** product_key (SK), id_product (source ID), product_name, validity_start_date, and validity_end_date;
- **dim_time:** time_key (SK), time_date (full date), time_day (day of the month), time_month (month), time_year (year), time_weekday (day of the week, as an integer), and time_quarter (quarter). This dimension does not have SCD since time is immutable;
- **fact_sales:** sale_key (SK, fact PK), seller_key, customer_key, product_key, and time_key (FKs referencing the respective dimensions, all NOT NULL), plus the metrics quantity, unit_price, total_price, and discount, also NOT NULL.

5.2. Time Dimension Population.

The dim_time dimension was populated using the file 2. Insert_Into_dim_time.sql. The strategy used was the programmatic generation of a continuous date range from January 1st, 1970 to December 31st, 2049, covering 29,220 days (indexes 0 to 29,219). The script is composed of three chained structures:

- The FROM block uses the GENERATE_SERIES(0, 29219) function to produce a sequence of integers that, added to the base date '1970-01-01'::DATE, generate the temporary datum

column with all dates in the range. This set is grouped by SEQUENCE.DAY to ensure each date appears exactly once;

- The SELECT block slices each datum value with the EXTRACT function, deriving the analytical attributes: time_day (day of the month), time_month (month), time_year (year), time_weekday (numeric day of the week, returned by the dow parameter, 0 for Sunday, 6 for Saturday), and time_quarter (quarter). The datum itself is assigned to the time_date field;
- The ORDER BY 1 block ensures records are inserted in ascending chronological order, from the oldest to the most recent date, maintaining the sequential integrity of the time dimension.

5.3. Data Load from OLTP to OLAP (SCD Type 2).

Data loads from the transactional to the analytical environment were performed using the file 3. Script.sql. The script was structured in sequential phases to demonstrate the complete operation of the Slowly Changing Dimensions (SCD) Type 2 mechanism.

5.3.1. Initial Load of Dimensions (customers, products, and sellers).

The same CTE pattern was applied to all three dimensions with SCD Type 2 support. The mechanism operates in three chained steps:

- **CTE S:** selects all records from the source table in the Relational schema (e.g., Relational.customers). This CTE acts as a mirror of the current state of the transactional source and serves as the basis for comparisons in subsequent steps;
- **CTE UPD:** executes an UPDATE on Dimensional.dim_customer (or dim_product / dim_seller). The filter T.id_customer = S.id_customer AND T.validity_end_date IS NULL locates, for each source ID, the single record still active, i.e., the one whose validity_end_date is still NULL. A second condition then checks whether any tracked attribute has changed (e.g., customer_status, customer_name). When a change is detected, validity_end_date receives current_date, "closing" that version of the record. The RETURNING T.id_customer clause returns the IDs of the newly closed records, making them available for the next step;
- **Final INSERT:** inserts the new records into the dimension. The WHERE condition captures two groups: (1) the IDs returned by the UPD CTE, which need a new active version, and (2) the IDs that do not yet exist in the dimension, completely new records. In both cases, validity_start_date receives current_date (marking the start of validity for the new version) and validity_end_date receives NULL (indicating this is the currently valid record).

5.3.2. Fact Table Load (month by month).

The fact_sales table is populated using an INSERT INTO ... SELECT that consolidates data from the transactional tables and analytical dimensions. Sales were loaded month by month (January, February, March, and subsequent months) to enable simulation and verification of SCD Type 2 between loads. The SELECT of the fact load operates with the following attribute and key mapping logic:

- **Metrics Mapping:** the attributes quantity, unit_price, total_price, and discount are extracted directly from Relational.sale_items iv, as they represent the detail data of each sold item, the granularity level of the fact table;

- **seller_key Mapping:** obtained through the INNER JOIN between Relational.sales v and Dimensional.dim_seller vdd using v.id_seller = vdd.id_seller AND vdd.validity_end_date IS NULL. The validity_end_date IS NULL filter ensures the sale is associated with the version of the seller that was active at the time of the load, not a closed historical version;
- **customer_key Mapping:** obtained through the INNER JOIN between Relational.sales v and Dimensional.dim_customer c using v.id_customer = c.id_customer AND c.validity_end_date IS NULL. The same validity_end_date IS NULL filter ensures that the SK recorded in the fact table points to the customer status active at the time that sale was loaded, the central principle of SCD Type 2;
- **product_key Mapping:** obtained through the INNER JOIN between Relational.sale_items iv and Dimensional.dim_product p using iv.id_product = p.id_product AND p.validity_end_date IS NULL. Again, the validity_end_date IS NULL filter captures only the active version of the product;
- **time_key Mapping:** obtained through the INNER JOIN between Relational.sales v and Dimensional.dim_time t using v.sale_date = t.time_date. The sale_date field from the transactional table is directly compared to the time_date field of the time dimension, locating the record that represents exactly the day the sale occurred, with no risk of duplication since the dimension has one unique record per date;
- **Time Filter:** the date_part('MONTH', v.sale_date) function extracts the month number from sale_date and filters only the sales of the desired month (e.g., = 01 for January, = 02 for February, > 03 for subsequent months).

5.3.3. Status Change Simulation and Historical Processing (SCD Type 2).

To demonstrate the operation of SCD Type 2 versioning, two status changes were simulated directly in the Relational.customers transactional table:

- **First change:** UPDATE Relational.customers SET customer_status = 'Gold' WHERE id_customer BETWEEN 1 AND 5, customers with IDs 1 to 5 had their customer_status promoted to 'Gold' after the January load;
- **Second change:** UPDATE Relational.customers SET customer_status = 'Platinum' WHERE id_customer = 3, the customer with ID 3 was promoted again to 'Platinum' after the March load.

After each change, the dimension load block (CTE S + CTE UPD + INSERT) was executed again. The mechanism detects the difference in customer_status, closes the previous record by setting its validity_end_date to current_date, and inserts a new row with validity_start_date = current_date and validity_end_date = NULL, thus preserving the complete version history of each customer.

5.3.4. Results Verification and SCD Type 2 Audit.

Throughout the script, audit queries were executed to validate the SCD Type 2 behavior. The SELECTs combine fact_sales with dim_customer (and, when necessary, with dim_time) via INNER JOIN using the respective SKs, filtering c.id_customer BETWEEN 1 AND 5. These queries prove that:

- January sales point to the old SKs of customers 1 to 5, i.e., to the version in which customer_status was not yet ‘Gold’. The historical link is preserved in the fact table even after the update in the dimension;

- Sales from February onwards, made after the status update, point to the new SK of each customer, reflecting the customer_status in effect at the time of that month's load;
- A second query filtering t.time_month = 3 confirms that in March, none of the customers with IDs 1 to 5 made purchases, validating that the fact table recorded no rows for this group in that period.

File used (1. Environments/2. OLAP/):

- 3. Script.sql.

5.4. Cube.

Since pgAdmin 4 is not a dimensional data server, a denormalized table was created that joins the fact table with all dimensions, forming the analytical cube. This process was carried out using the file 4. Cube.sql and is composed of three main structures:

- The SELECT block chooses only the attributes relevant for analysis, deliberately excluding the Surrogate Keys (seller_key, customer_key, product_key, time_key), the source IDs (id_seller, id_customer, id_product), and the versioning fields (validity_start_date, validity_end_date). The selected attributes are: customer_name, customer_state, customer_gender, and customer_status from dim_customer; product_name from dim_product; time_date, time_day, time_month, time_year, and time_quarter from dim_time; seller_name from dim_seller; and the metrics quantity, unit_price, total_price, and discount from fact_sales;
- The INTO Dimensional.sales_cube block physically creates the destination table in the Dimensional schema, consolidating in a single flat structure all the data needed for Power BI analysis;
- The FROM block starts from the central table Dimensional.fact_sales fv and connects all dimensions through INNER JOINs: dim_customer dc via fv.customer_key = dc.customer_key; dim_product dp via fv.product_key = dp.product_key; dim_time dt via fv.time_key = dt.time_key; and dim_seller dv via fv.seller_key = dv.seller_key. The use of INNER JOIN ensures that only valid combinations, where all dimensions have a corresponding record, make up the cube.

File used (1. Environments/2. OLAP/):

- 4. Cube.sql.

5.5. KPI (Key Performance Indicator).

To measure the total revenue of JJBike against monthly targets, the Dimensional.kpi table was created using the file 5. KPI.sql. The script is composed of four main structures:

- The SELECT block defines the three columns that will make up the table: (1) dt.time_month aliased as month, which represents the month number; (2) SUM(fv.total_price) aliased as total_revenue, which aggregates the revenue earned by summing the total_price field of all fact_sales records belonging to that month; and (3) a CASE structure dt.time_month that assigns a fixed target value to each month (from R$ 220,000 in January to R$ 270,000 in December), aliased as target;
- The ::numeric cast operator, placed after the END of the CASE block, forces the target column to be consolidated as an exact decimal numeric type, the same type as total_price in

fact_sales. Without this conversion, the column dynamically generated by the CASE could be treated as an integer type by PostgreSQL, causing format incompatibility when calculating achievement percentages in Power BI;

- The INTO Dimensional.kpi block physically creates the target table in the Dimensional schema, storing the consolidated realized and target revenue data by month;
- The FROM block starts from Dimensional.fact_sales fv and performs an INNER JOIN with Dimensional.dim_time dt using fv.time_key = dt.time_key, which relates each fact table row to its corresponding record in the time dimension, providing the time_month field needed for grouping. The GROUP BY dt.time_month consolidates all sales of each month into a single row, and ORDER BY dt.time_month ASC organizes the result in ascending chronological order.

File used (1. Environments/2. OLAP/):

- 5. KPI.sql.

6. Artifacts – Power BI.

1. The tables from the *Dimensional* database, created in pgAdmin 4, were connected to Power BI.

6.1. Reports.

1. The following changes were made to the tables:

- In fact_sales, a column called discount_percent (discount percentage) was created (with two decimal places), with the following formula: discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2);
- In dim_customer, a column called full_location (full location) was created, with the following formula:
    - location_map =
      SWITCH(
        'dimensional dim_customer'[customer_state],
        "AC", "Acre",
        "AL", "Alagoas",
        "AM", "Amazonas",
        "AP", "Amapa",
        "BA", "Bahia",
        "CE", "Ceara",
        "DF", "Federal District",
        "ES", "Espirito Santo",
        "GO", "Goias",
        "MA", "Maranhao",
        "MG", "Minas Gerais",
        "MS", "Mato Grosso do Sul",
        "MT", "Mato Grosso",
        "PA", "Para",
        "PB", "Paraiba",
        "PE", "Pernambuco",
        "PI", "Piauhi",

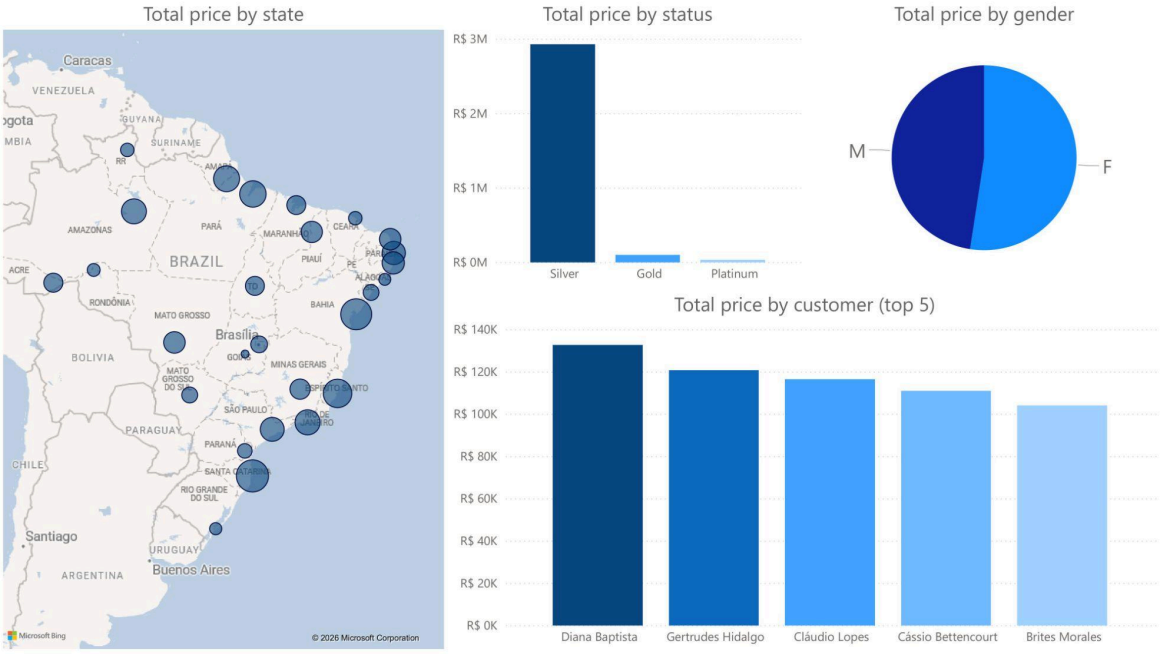
```
        "PR", "Parana",
        "RJ", "Rio de Janeiro",
        "RN", "Rio Grande do Norte",
        "RO", "Rondonia",
        "RR", "Roraima",
        "RS", "Rio Grande do Sul",
        "SC", "Santa Catarina",
        "SE", "Sergipe",
        "SP", "Sao Paulo",
        "TO", "Tocantins",
        "Unknown"
    ) & ", Brazil".
```

- In dim_kpi, three metrics were created so that, in the KPI report, the card has dynamic behavior according to what is selected in the button slicer:
    - measure_target =
      ```
      IF(
          ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
          SUM('dimensional kpi'[target]),
          0
      ).
      ```
    - measure_total_revenue =
      ```
      IF(
          ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
          SUM('dimensional kpi'[total_revenue]),
          0
      ).
      ```

2. The following reports were created:

(Customers)

The customer report presents: total value sum by state, total value sum by status, total value sum by gender, and total value sum by customer (top 5).



(Sellers)

The seller report presents: total value sum by seller (top 5 best sellers, top 5 worst sellers, and top 5 best sellers compared month by month).

(Products)

The product report presents: total value sum by product (top 5 best-selling products and top 5 least-selling products) and total discount sum by product (top 5 products with the highest accumulated discount and top 5 products with the lowest accumulated discount).

6.2. Dashboards.

1. The following dashboards were created:

(Sales)

The sales dashboard presents: total value sum of products sold (compared month by month), total quantity of products sold (compared month by month), total discount of products sold (compared month by month), and product, seller, and customer cards for filtering.

(KPI)

The KPI dashboard presents: target, total acquired value, target sum and total acquired value (compared month by month), and month and year cards for filtering.

6.3. Cube.

1. The following hierarchies were created from the dimensions:

- **Geographic Hierarchy:** customer_state → customer_name (dim_customer);
- **Time Hierarchy:** time_year → time_quarter → time_month → time_day (dim_time);
- **Notes:**
  - customer_gender and customer_status must remain free in the side panel as Loose Dimension Attributes, since inserting them in the Geographic Hierarchy would imply they are part of a geographic subdivision;
  - dim_seller and dim_product have only one attribute beyond their keys, therefore they did not generate hierarchies and their attributes became Loose Dimension Attributes.

2. Visual representation of the created cube:

Customer State	Quantity	Discount	Total Price
AC	34	R$ 11,172.65	R$ 88,878.08
AL	6	R$ 4,199.33	R$ 26,817.61
AM	57	R$ 15,881.10	R$ 168,571.45
AP	53	R$ 16,202.60	R$ 183,945.80
BA	65	R$ 18,104.59	R$ 274,690.93
CE	11	R$ 9,146.31	R$ 37,562.17
DF	19	R$ 14,839.02	R$ 65,272.18
ES	89	R$ 12,590.55	R$ 226,925.84
GO	3	R$ 858.20	R$ 7,614.38
MA	38	R$ 5,149.81	R$ 87,431.06
MG	31	R$ 18,853.70	R$ 102,980.97
MS	19	R$ 11,094.30	R$ 57,340.96
MT	44	R$ 11,279.89	R$ 119,272.51
PA	82	R$ 24,476.00	R$ 190,925.91
PB	51	R$ 8,791.16	R$ 143,634.26
PE	42	R$ 19,477.64	R$ 129,473.93
PI	44	R$ 20,851.47	R$ 116,457.99
PR	22	R$ 5,322.31	R$ 45,718.33
RJ	57	R$ 19,318.44	R$ 169,913.05
RN	45	R$ 16,227.19	R$ 118,559.35
RO	13	R$ 6,639.07	R$ 33,213.18
RR	28	R$ 7,962.75	R$ 37,392.82
RS	12	R$ 2,768.23	R$ 28,466.43
SC	91	R$ 35,968.90	R$ 296,803.85
SE	15	R$ 11,054.02	R$ 56,151.97
SP	42	R$ 17,043.43	R$ 152,112.79
TO	47	R$ 4,163.74	R$ 88,034.93
Total	**1060**	**R$ 349,436.40**	**R$ 3,054,162.73**

Generated files available in '2. Artifacts/'.